

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS COURSE CODE: CSA0711

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools.

(BL4,BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/ modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. NETWORK THROUGHPUT AND PACKET LOSS CALCULATION

1.1 Problem Statement

In a network simulation using Cisco Packet Tracer, a student observes that **8000 bits of data were received in 4 seconds** . During the simulation, **200 packets were sent and 180 packets were received** .

The objective is to develop a pseudocode algorithm that calculates:

- Network throughput in bits per second.
- Packet loss percentage.
- Network efficiency based on predefined conditions.

The network is classified as **Efficient** when:

- Throughput > 1500 bps
- Packet loss < 10%

If either condition is not satisfied, the network is classified as **Inefficient** .

The algorithm should also validate the input values to prevent division-by-zero errors.

1.2 Given Values

Parameter	Value
Total Data Received	8000 bits
Time	4 seconds
Total Packets Sent	200
Total Packets Received	180
Throughput Threshold	1500 bps
Packet Loss Threshold	10%

1.3 Formula Used

Throughput

Throughput represents the amount of successfully received data per unit of time.

Throughput = Total Data Received / Time

Therefore:

Throughput = 8000 / 4 = 2000 bps

Packet Loss

First, calculate the number of lost packets:

Packet Loss = Packets Sent – Packets Received

Packet Loss = 200 – 180 = 20 packets

Packet loss percentage is:

Packet Loss % = (Packets Lost / Packets Sent) × 100

Packet Loss % = (20 / 200) × 100 = 10%

1.4 Pseudocode Algorithm

ALGORITHM NetworkPerformanceAnalysis

BEGIN

 // Input values

 totalDataReceived ← 8000

 time ← 4

 totalPacketsSent ← 200

 totalPacketsReceived ← 180

 // Validate time

 IF time = 0 THEN

 DISPLAY "Error: Time cannot be zero."

```

    STOP
END IF

// Validate total packets sent
IF totalPacketsSent = 0 THEN
    DISPLAY "Error: Total packets sent cannot be zero."
    STOP
END IF

// Calculate throughput
throughput ← totalDataReceived / time

// Calculate packet loss
packetsLost ← totalPacketsSent - totalPacketsReceived

// Calculate packet loss percentage
packetLossPercentage ← (packetsLost / totalPacketsSent) × 100

// Display calculated results
DISPLAY "----- NETWORK PERFORMANCE SUMMARY -----"
DISPLAY "Total Data Received: ", totalDataReceived, " bits"
DISPLAY "Time: ", time, " seconds"
DISPLAY "Total Packets Sent: ", totalPacketsSent
DISPLAY "Total Packets Received: ", totalPacketsReceived
DISPLAY "Packets Lost: ", packetsLost
DISPLAY "Throughput: ", throughput, " bps"
DISPLAY "Packet Loss: ", packetLossPercentage, "%"

// Classify network
IF throughput > 1500 AND packetLossPercentage < 10 THEN
    networkStatus ← "Efficient"
ELSE
    networkStatus ← "Inefficient"
END IF

DISPLAY "Network Classification: ", networkStatus

// Final summary
DISPLAY "----- FINAL RESULT -----"
DISPLAY "Throughput = ", throughput, " bps"
DISPLAY "Packet Loss = ", packetLossPercentage, "%"
DISPLAY "Network Status = ", networkStatus

```

END

1.5 Step-by-Step Working

Step 1: Validate the time

The algorithm checks whether the time is zero.

IF time = 0 THEN

 DISPLAY "Error"

 STOP

END IF

This validation is required because throughput involves division by time. Division by zero would produce an error.

Here:

Time = 4 seconds

Therefore, the calculation is valid.

Step 2: Validate packets sent

The algorithm checks whether the total number of packets sent is zero.

Here:

Packets Sent = 200

Therefore, packet loss percentage can be calculated safely.

Step 3: Calculate throughput

throughput = $8000 / 4$

 = 2000 bps

Therefore, the network throughput is **2000 bits per second**.

Step 4: Calculate packets lost

Packets Lost = $200 - 180$

 = 20 packets

Step 5: Calculate packet loss percentage

Packet Loss % = $(20 / 200) \times 100$

 = 10%

Step 6: Classify the network

The conditions are:

Throughput > 1500 bps

AND

Packet Loss < 10%

The calculated values are:

Throughput = **2000 bps** → condition satisfied.

Packet Loss = **10%** → condition **not** satisfied because the requirement is *less than* 10%.

Therefore:

Network Classification = Inefficient

1.6 Expected Output

---- NETWORK PERFORMANCE SUMMARY ----

Total Data Received: 8000 bits

Time: 4 seconds

Total Packets Sent: 200

Total Packets Received: 180

Packets Lost: 20

Throughput: 2000 bps

Packet Loss: 10%

Network Classification: Inefficient

---- FINAL RESULT ----

Throughput = 2000 bps

Packet Loss = 10%

Network Status = Inefficient

1.7 Advantages of the Algorithm

1. Calculates throughput automatically.
2. Calculates packet loss percentage accurately.
3. Prevents division-by-zero errors through validation.
4. Provides a clear network classification.
5. Can be easily modified for real-time network monitoring.
6. Can be implemented in different programming languages.
7. Helps identify whether the network meets performance requirements.

1.8 Conclusion

The algorithm calculates the network throughput as **2000 bps** and packet loss as **10%**. Although the throughput is greater than the required 1500 bps, the packet loss is exactly 10%, whereas the requirement is less than 10%. Hence, the network is classified as **Inefficient** according to the given conditions. The validation checks make the algorithm safer by preventing invalid division operations.

2. BANDWIDTH UTILIZATION MONITORING AND OVERLOAD DETECTION

2.1 Problem Statement

An organization has multiple network links connecting its branches. The network administrator wants to continuously monitor bandwidth utilization and identify links that are becoming overloaded.

The algorithm should:

- Periodically collect bandwidth usage.
- Store the values in an array or list.

- Monitor all available network links using a loop.
- Check whether utilization exceeds **80%**.
- Recommend switching traffic to a backup link when utilization exceeds 80%.
- Continue monitoring periodically.

This approach helps prevent network congestion and improves bandwidth management.

2.2 Example Network Links

Assume the organization has five network links.

Link	Maximum Bandwidth	Current Utilization
Link 1	100 Mbps	60%
Link 2	200 Mbps	85%
Link 3	150 Mbps	70%
Link 4	100 Mbps	90%
Link 5	250 Mbps	55%

The utilization values are stored in an array:
utilization = [60, 85, 70, 90, 55]

2.3 Pseudocode Algorithm

ALGORITHM BandwidthMonitoring

BEGIN

```
// Store network link information
links ← ["Link1", "Link2", "Link3", "Link4", "Link5"]

// Arrays to store bandwidth information
bandwidth ← [100, 200, 150, 100, 250]
utilization ← [60, 85, 70, 90, 55]
```

WHILE TRUE DO

```
    DISPLAY "---- BANDWIDTH MONITORING ----"

    // Monitor every network link
    FOR i ← 0 TO LENGTH(links) - 1 DO
```

```

DISPLAY "Monitoring ", links[i]

// Collect current bandwidth utilization
currentUtilization ← utilization[i]

DISPLAY "Bandwidth: ", bandwidth[i], " Mbps"
DISPLAY "Utilization: ", currentUtilization, "%"

// Check for overload
IF currentUtilization > 80 THEN

    DISPLAY "WARNING: ", links[i], " is overloaded."
    DISPLAY "Utilization exceeds 80%."

    DISPLAY "Recommendation:"
    DISPLAY "Switch traffic to backup link."

    // Traffic switching action
    SWITCH_TRAFFIC_TO_BACKUP(links[i])

ELSE

    DISPLAY links[i], " is operating normally."

END IF

END FOR

DISPLAY "Monitoring cycle completed."

// Wait before next monitoring cycle
WAIT 10 seconds

END WHILE

END

```

2.4 Step-by-Step Explanation

Step 1: Store link information

The network links are stored in an array.

links = [Link1, Link2, Link3, Link4, Link5]

Bandwidth capacity is also stored.

bandwidth = [100, 200, 150, 100, 250]

The utilization values are stored separately.

utilization = [60, 85, 70, 90, 55]

This makes it easier to monitor multiple links using a single loop.

Step 2: Start continuous monitoring

The algorithm uses:

WHILE TRUE

This allows the administrator to continuously monitor the network.

After completing one monitoring cycle, the system waits for a fixed period before collecting the next set of values.

For example:

Monitoring interval = 10 seconds

Step 3: Monitor each link

A FOR loop checks every link.

FOR i ← 0 TO LENGTH(links) - 1

Therefore, Link 1, Link 2, Link 3, Link 4, and Link 5 are checked one after another.

Step 4: Check utilization

The algorithm compares utilization with the threshold:

IF currentUtilization > 80

If utilization is greater than 80%, the link is considered overloaded.

For example:

Link 2 = 85%

Since:

85% > 80%

Link 2 is overloaded.

Similarly:

Link 4 = 90%

Since:

90% > 80%

Link 4 is also overloaded.

Step 5: Recommend backup link

When an overloaded link is detected, the algorithm displays:

WARNING: Link is overloaded.

Recommendation: Switch traffic to backup link.

Traffic can then be redirected to an available backup link.

2.5 Example Output

----- BANDWIDTH MONITORING -----

Monitoring Link1

Bandwidth: 100 Mbps

Utilization: 60%

Link1 is operating normally.

Monitoring Link2

Bandwidth: 200 Mbps

Utilization: 85%

WARNING: Link2 is overloaded.

Utilization exceeds 80%.

Recommendation: Switch traffic to backup link.

Monitoring Link3

Bandwidth: 150 Mbps

Utilization: 70%

Link3 is operating normally.

Monitoring Link4

Bandwidth: 100 Mbps

Utilization: 90%

WARNING: Link4 is overloaded.

Utilization exceeds 80%.

Recommendation: Switch traffic to backup link.

Monitoring Link5

Bandwidth: 250 Mbps

Utilization: 55%

Link5 is operating normally.

Monitoring cycle completed.

2.6 How the Algorithm Supports Efficient Bandwidth Management

1. Continuous Monitoring

The algorithm continuously observes the network instead of checking the links manually.

2. Early Overload Detection

The 80% threshold provides an early warning before a link becomes completely congested.

3. Backup Traffic Switching

Traffic can be redirected to a backup link when the primary link becomes overloaded.

4. Reduced Congestion

Moving traffic away from highly utilized links reduces congestion and improves network performance.

5. Better Resource Utilization

Available bandwidth on other links can be used more effectively.

6. Scalability

New links can be added to the arrays without significantly changing the algorithm.

7. Automated Management

The monitoring process can be automated, reducing the administrator's manual workload.

2.7 Conclusion

The proposed bandwidth monitoring algorithm continuously checks the utilization of all network links and identifies links whose utilization exceeds **80%**. When an overloaded link is detected, the algorithm recommends switching traffic to a backup link. This prevents congestion, improves network performance, increases reliability, and allows the organization to use its available bandwidth efficiently.

3. DYNAMIC BEST-PATH SELECTION BASED ON LINK QUALITY

3.1 Problem Statement

A company has six branch offices connected through multiple network links. Each link has different values for:

Bandwidth

Delay

Utilization

These values change throughout the day.

The administrator wants to continuously identify the **best communication path** between headquarters and each branch based on overall link quality rather than simply selecting the shortest path.

The algorithm should:

1. Periodically collect bandwidth, delay, and utilization.
2. Store the values in arrays or lists.
3. Monitor the network continuously.
4. Calculate a link quality score.
5. Select the path with the highest score.
6. Detect overloaded links.
7. Detect failed links.
8. Automatically select an alternate path.
9. Generate an alert for the administrator.

3.2 Network Parameters

Three important parameters are considered.

Bandwidth

Bandwidth represents the maximum data-carrying capacity of a link.

Higher bandwidth → Better link quality

Delay

Delay represents the time required for data to travel across the network.

Lower delay → Better link quality

Utilization

Utilization indicates how much of the available bandwidth is currently being used.

Lower utilization → Better link quality

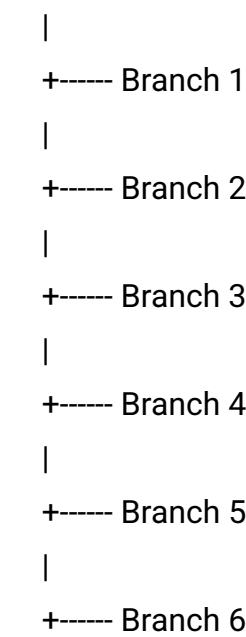
Therefore, a good link should generally have:

High bandwidth + Low delay + Low utilization

3.3 Example Network

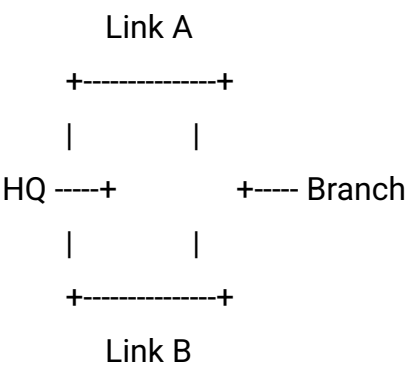
Assume the company has six branches:

Headquarters



There may be multiple possible paths between headquarters and each branch.

For example:



The algorithm evaluates the available paths instead of choosing a path only based on distance.

3.4 Arrays Used

The algorithm can maintain arrays such as:

- bandwidth[]
- delay[]
- utilization[]
- status[]
- qualityScore[]

For example:

bandwidth = [100, 200, 150, 100, 250]

delay = [20, 30, 15, 50, 25]

utilization = [50, 70, 40, 90, 60]

status = ["UP", "UP", "UP", "UP", "UP"]

3.5 Link Quality Score

A combined score can be used to compare different links.

First, the parameters should be normalized because bandwidth, delay, and utilization have different units.

A simple conceptual scoring method is:

Quality Score =

(Bandwidth Score $\times$ 0.5)

+ (Delay Score $\times$ 0.3)

+ (Utilization Score $\times$ 0.2)

The weights can be changed according to the organization's requirements.

In this example:

Bandwidth receives **50% importance**.

Delay receives **30% importance**.

Utilization receives **20% importance**.

The score rewards high bandwidth while preferring low delay and low utilization.

3.6 Pseudocode Algorithm

ALGORITHM DynamicBestPathSelection

BEGIN

// List of branches

branches $\leftarrow$ ["Branch1", "Branch2", "Branch3",
"Branch4", "Branch5", "Branch6"]

// Monitoring interval

monitoringInterval $\leftarrow$ 10 seconds

WHILE TRUE DO

DISPLAY "===== NETWORK PATH MONITORING ====="

// Monitor every branch

FOR each branch IN branches DO

DISPLAY "Monitoring paths to ", branch

```

// Discover all available paths
paths ← GET_AVAILABLE_PATHS(branch)

bestPath ← NULL
highestScore ← -1

// Evaluate every path
FOR each path IN paths DO

    // Collect current network parameters
    bandwidth ← GET_BANDWIDTH(path)
    delay ← GET_DELAY(path)
    utilization ← GET_UTILIZATION(path)
    status ← GET_STATUS(path)

    // Store collected values
    STORE bandwidth
    STORE delay
    STORE utilization
    STORE status

    // Check whether link has failed
    IF status = "DOWN" THEN

        DISPLAY "Link failure detected on ", path
        GENERATE_ALERT("Link failure", path)

        CONTINUE

    END IF

    // Check whether link is overloaded
    IF utilization > 80 THEN

        DISPLAY "Warning: ", path, " is overloaded."
        GENERATE_ALERT("Link overloaded", path)

        // Do not select overloaded path
        CONTINUE

    END IF

```

```

// Calculate normalized scores
bandwidthScore ← NORMALIZE_BANDWIDTH(bandwidth)
delayScore ← NORMALIZE_DELAY(delay)
utilizationScore ← NORMALIZE_UTILIZATION(utilization)

// Calculate overall quality score
qualityScore ←
    (bandwidthScore × 0.5) +
    (delayScore × 0.3) +
    (utilizationScore × 0.2)

DISPLAY "Path: ", path
DISPLAY "Bandwidth: ", bandwidth
DISPLAY "Delay: ", delay
DISPLAY "Utilization: ", utilization
DISPLAY "Quality Score: ", qualityScore

// Select highest quality path
IF qualityScore > highestScore THEN

    highestScore ← qualityScore
    bestPath ← path

END IF

END FOR

// Check if a suitable path exists
IF bestPath ≠ NULL THEN

    SELECT_PATH(branch, bestPath)

    DISPLAY "Best Path for ", branch, ": ", bestPath
    DISPLAY "Quality Score: ", highestScore

ELSE

    DISPLAY "No suitable path available for ", branch
    GENERATE_ALERT("No available path", branch)

END IF

```

END FOR

DISPLAY "===== MONITORING CYCLE COMPLETED ====="

WAIT monitoringInterval

END WHILE

END

3.7 Explanation of the Algorithm

Step 1: Identify Branches

The six branch offices are stored in a list.

branches = [Branch1, Branch2, Branch3,
 Branch4, Branch5, Branch6]

This allows the same monitoring procedure to be applied to every branch.

Step 2: Obtain Available Paths

For each branch, the algorithm identifies all possible communication paths.

For example:

HQ → Router A → Router B → Branch 1

HQ → Router C → Router D → Branch 1

Both paths are evaluated.

Step 3: Collect Link Information

For every path, the algorithm collects:

Bandwidth

Delay

Utilization

Status

These values are stored in appropriate arrays or lists.

For example:

Bandwidth = [100, 200, 150]

Delay = [20, 35, 15]

Utilization = [50, 75, 40]

Step 4: Check Link Failure

The algorithm first checks the link status.

IF status = "DOWN"

If the link has failed:

The path is not selected.
An alert is generated.
The algorithm evaluates another available path.

Example:
ALERT: Link failure detected.
Path removed from selection.
Searching for alternate path...

Step 5: Detect Overloaded Links

The algorithm checks:
IF utilization > 80
If utilization exceeds 80%, the link is considered overloaded.
The algorithm:
1. Generates a warning.
2. Generates an alert.
3. Avoids selecting the overloaded path.
4. Searches for another suitable path.
This helps prevent congestion.

3.8 Example

Suppose three paths are available between headquarters and Branch 1.

Path	Bandwidth	Delay	Utilization	Status
Path A	100 Mbps	20 ms	50%	UP
Path B	200 Mbps	35 ms	75%	UP
Path C	150 Mbps	15 ms	40%	UP

Path B has the highest bandwidth, but it also has higher delay and utilization.
Path C has:
Good bandwidth
Lowest delay
Lowest utilization

Therefore, after calculating the overall quality score, **Path C may be selected as the best path.**
This demonstrates why considering multiple metrics is better than simply selecting the path with the highest bandwidth or shortest distance.

3.9 Handling an Overloaded Path

Suppose Path C changes to:

Utilization = 90%

The algorithm detects:

$90\% > 80\%$

Therefore:

WARNING: Path C is overloaded.

ALERT: Traffic should be moved to an alternate path.

The algorithm excludes Path C and evaluates Paths A and B.

The best remaining path is then selected.

3.10 Handling a Link Failure

Suppose Path A changes from:

Status = UP

to:

Status = DOWN

The algorithm detects the failure and generates:

ALERT: Path A has failed.

Searching for alternate path...

The failed path is removed from consideration.

The algorithm then selects the best available path from the remaining links.

This provides **automatic fault tolerance**.

3.11 Monitoring at Regular Intervals

Network conditions can change throughout the day.

For example:

10:00 AM → Utilization = 40%

12:00 PM → Utilization = 65%

02:00 PM → Utilization = 85%

04:00 PM → Utilization = 55%

Therefore, selecting a path only once is not sufficient.

The algorithm uses:

WAIT 10 seconds

and then repeats the monitoring process.

This allows the routing decision to adapt to changing network conditions.

3.12 Advantages of the Algorithm

1. Improved Network Reliability

The algorithm continuously monitors network links and automatically avoids failed links.

Therefore, communication can continue through an alternate path.

2. Better Routing Efficiency

Instead of selecting a path only based on distance, the algorithm considers:

- Bandwidth

- Delay

Utilization

Link status

This produces better routing decisions.

3. Fault Tolerance

When a link fails, the algorithm automatically searches for another available path.

This reduces service interruption.

4. Congestion Prevention

Links with utilization above 80% are avoided.

This helps prevent excessive network congestion.

5. Dynamic Adaptation

Because network parameters are collected periodically, the algorithm adapts to changing network conditions.

6. Better Resource Utilization

Traffic can be distributed toward links with better available capacity.

7. Administrator Alerts

The administrator receives alerts when:

A link fails.

A link becomes overloaded.

No suitable path is available.

This allows timely corrective action.

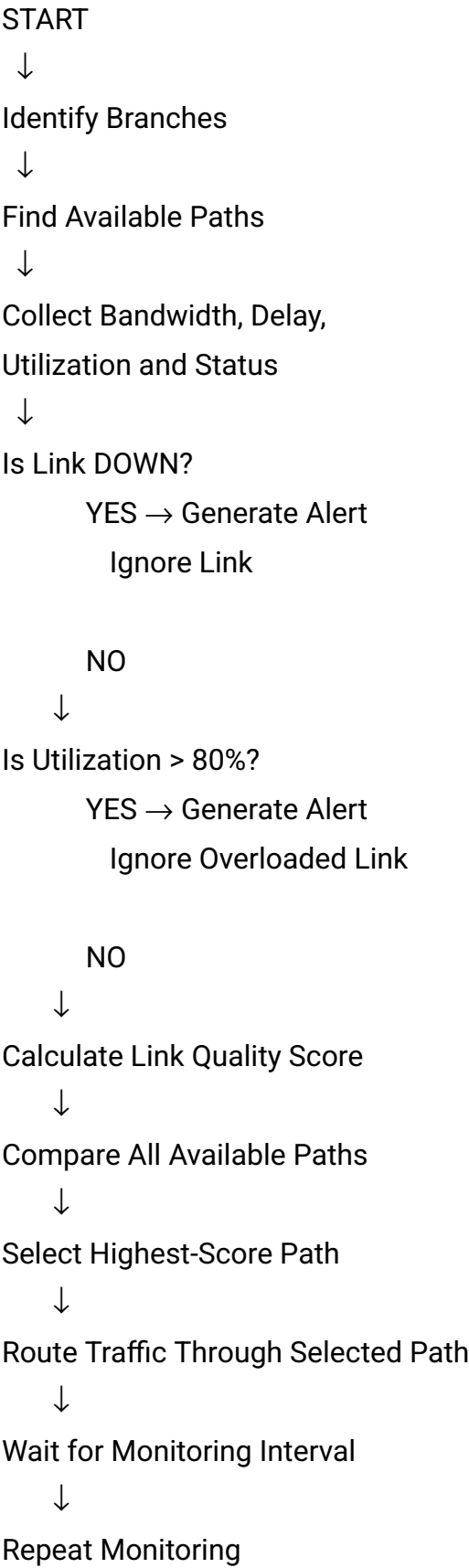
8. Scalability

The same algorithm can be extended to additional branches and network links by adding them to the corresponding lists.

3.13 Requirement-to-Solution Mapping

Requirement	Solution
Six branch offices	Branch list and loop
Multiple network links	Available-path list
Changing bandwidth	Periodic bandwidth collection
Changing delay	Periodic delay collection
Changing utilization	Continuous utilization monitoring
Best path selection	Quality score
Overloaded link	80% utilization threshold
Link failure	Status checking
Alternate path	Dynamic path selection
Administrator notification	Alert generation
Continuous operation	Repeated monitoring loop

3.14 Overall Working Flow



3.15 Conclusion

The proposed dynamic path-selection algorithm provides an intelligent method for selecting communication paths between headquarters and the six branch offices. Instead of considering only the shortest path, it evaluates **bandwidth, delay, utilization, and link status** to calculate an overall link quality score.

The algorithm continuously monitors the network at regular intervals. If a link becomes overloaded or fails, it generates an alert and automatically searches for an alternate path. This improves **network reliability, routing efficiency, congestion control, and fault tolerance**.

Since the network conditions change throughout the day, continuous monitoring allows the system to adapt its routing decisions dynamically. Therefore, the proposed approach provides a scalable and efficient solution for managing a multi-branch enterprise network.